

Development of a Scalable Industrial Process Monitoring Sensor System for large-scale IoT Networks using MQTT and Time Series Database

Samiul Hoque, Imtiaz Ahmmed, Asif Jobair, Md Shakhawat Hossain*, Mohammad Rejwan Uddin, and Mahady Hasan

*Fab Lab IUB and RIoT Lab IUB**
Independent University, Bangladesh

Dhaka, Bangladesh

Email: s026hoque@gmail.com; imtiazahmmed00@gmail.com; asifjobair30@gmail.com; shakhawat@iub.edu.bd; rejwan@iub.edu.bd; mahady@iub.edu.bd.

Abstract— This research paper presents the design process and implementation of a large-scale industrial Process Monitoring Sensor System for large-scale IoT deployments. With the rising popularity of IoT devices, a major concern is the ease of use and integration of these off-the-shelf devices into the existing infrastructure of embedded systems. This paper focuses on building easy-to-use back end and front-end software that can be interfaced to any sensor that uses the widely popular MQTT protocol. By utilizing a Time Series database to store the acquired data, the system can deliver precise range queries which can produce powerful data visualizations for key insights. The proposed system can be hosted on any cloud service provider and was tested extensively with both off-the-shelf commercially available sensor modules and sensors developed using the ESP32 microcontroller series. All peripherals used are commonly used in industrial cases and the sensors use Wi-Fi (IEEE 802.11b/g). The experiment demonstrates successful data acquisition of both telemetry and events from five different sensor modules into a time series database.

Keywords— *IoT System, Software Architecture, Hierarchical hardware topology, MQTT, Time Series Database, Data Acquisition, SCADA, large-scale IoT systems*

I. INTRODUCTION

Embedded systems are a key component of almost all industrial automation processes these days. Moreover, with the increased availability of smart devices in our day to day lives, the IoT revolution is now shifting towards data acquisition from big data sources to provide business intelligence. A simple toaster can now be connected to a backend server and provide analytics towards user behavior such as daily bread intake, breakfast timing and energy consumption [1]. We can imagine this problem with a scale for millions of devices and we can start seeing the complexity and scalability issue of conventional IoT data acquisition software.



Fig. 1. Example of a Large Scale IoT network

In this paper, we will be focusing mostly on the development and hierarchical architecture that can be used to solve the scalability and complexity issues of a modern IoT backend as shown in Fig. 1 [2].

This paper is organized as follows. Section II describes the system architecture while explaining the technical specifications of each core component. Section III explains the topology we have implemented in our specific system. In section IV we test and evaluate the performance of the proposed system using a simple simulation script. And later in Section V we briefly describe the five hardware components we have developed to use in our system. Finally, in Section VI, we evaluate our system following up with presenting our conclusion in Section VII.

II. PROPOSED SYSTEM TOPOLOGY

The proposed system topology has three primary layers; 1. Physical Layer 2. Network Layer 3. Software Abstraction Layer. Fig. 2 describes the three layers and their components of the proposed system.

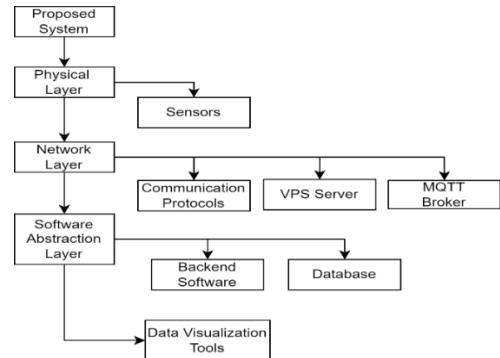


Fig. 2. Topology of proposed system

The details of the core technical components of the system are described in bottom-up order in this section.

A. Backend Software

The backend is built using Node-Red. Considering the ease of use for the end user, we have chosen node-red for the backend application. The MQTT message is first received and then it is pushed into an encrypted InfluxDB Time Series database. Fig. 3 describes the transmission of data from the Sensors to the InfluxDB database [3].

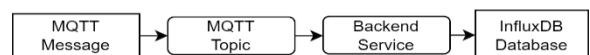


Fig. 3. MQTT dataflow in the proposed system

This flow (Fig. 3) can be achieved simply with the following flow in node-red which is shown in Fig. 4



Fig. 4. Node-Red Flow for MQTT data acquisition into InfluxDB.

For overly complicated systems, the flow just adds each component as required by the system, but the underlying logic remains the same.

B. Database

For the Database, we are using InfluxDB v1.8.10. This specific version is open source while InfluxDB v2.x+ has paid features and uses flux as the query language. The advantage of using a time series database over a relational database in IoT applications is that it takes away the problem of complicated schemas and instead stores all data only with reference to the timestamp it receives the data in [4].

```
sensi@monitoring-tools:~$ influx
Connected to http://localhost:8086 version 1.8.10
InfluxDB shell version: 1.8.10
> use test
Using database test
> show measurements
name: measurements
name
----
sensi/Sensor_001/events
sensi/Sensor_001/telemetry
sensi/Sensor_001/telemetry/TDS
sensi/Sensor_001/telemetry/temperature
sensi/Sensor_002/telemetry/altitude
sensi/Sensor_002/telemetry/pressure
sensi/Sensor_002/telemetry/temperature
> SELECT * FROM "sensi/Sensor_001/telemetry" LIMIT 5
name: sensi/Sensor_001/telemetry
time                value
----
1669177808086434945 211
1669177810178725342 212
1669177812288048241 213
1669177814395349484 214
1669177816516262233 215
```

Fig. 5. Example of InfluxDB data storage

As demonstrated in Fig. 5 we can see that InfluxDB stores all data from a specific topic as measurements under a single database. And then sequentially stores payload data under that measurement only with a reference to its time stamp.

C. Data Visualization Tool

There are multiple tools which can be used to visualize data from an InfluxDB database. For the scope of this project, we did not build our own front end but used an industry standard query visualization tool called Grafana which is used widely in System OPs and DevOps for process monitoring [5].

D. Communication Protocols

For the scope of our project, all the wireless communication is done in 2.4GHz Wi-Fi while the message payload is delivered through web sockets to the MQTT broker [6]. The MQTT messages are encrypted to make the system more secure against man-in-the-middle attacks.

E. VPN Server

We are using Digital Ocean for our VPS provider. The system is running Ubuntu 22.10 with the system specifications of 4 vCPUs, 8GB RAM, and 160GB SSD and is in Toronto.

F. MQTT Broker

The MQTT broker is at the heart of the proposed system. We are running Mosquitto v2.09 which supports MQTT v5.0/v3.1.1/v3.1 [7]. In our Mosquitto configuration, we have implemented MQTT over web sockets which is more secure than conventional MQTT and uses end-to-end encryption of topics & messages.

G. Sensors

To demonstrate the scalability of the proposed system, five sensors are being used of which four are self-fabricated and one is bought off the shelf shown in Fig. 6. Each sensor can send two types of data. One being the telemetry data which is the sensor data being sent repeatedly over time [8].

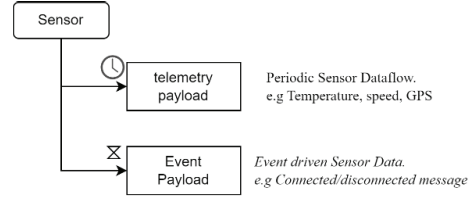


Fig. 6. Sensor data types in the proposed Hardware Layer

The other one is event data where it sends connection/disconnection events as payload.

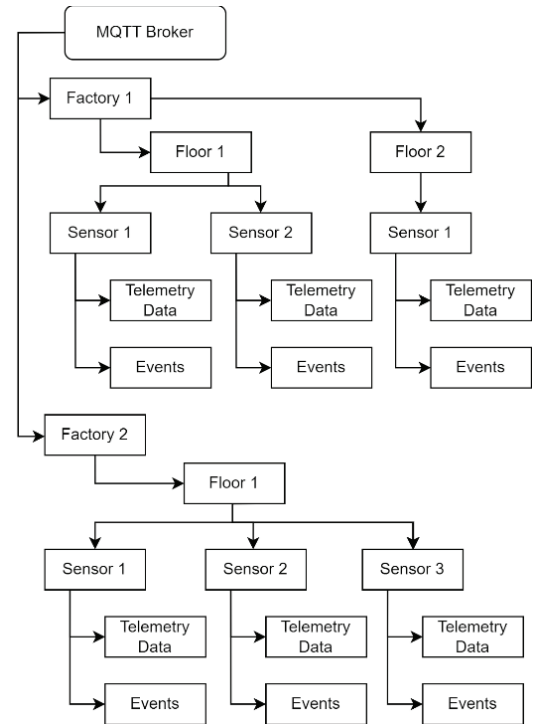


Fig. 7. Example of hierarchical complexity that can be achieved using MQTT topic as class identifiers.

This structure allows for asynchronous polling of sensor data while we can also keep track of if any of the sensors are malfunctioning or not.

III. PROPOSED SYSTEM METHODOLOGY

The proposed system has been designed to demonstrate the scalable properties of the hierarchical architecture shown in Fig. 7 and the discussed System Topology in Section II

A. System Architecture

In Fig. 8 we have implemented all the core components of the system discussed in Section II along with five demonstrable sensor devices which are later discussed in Section IV of this paper.

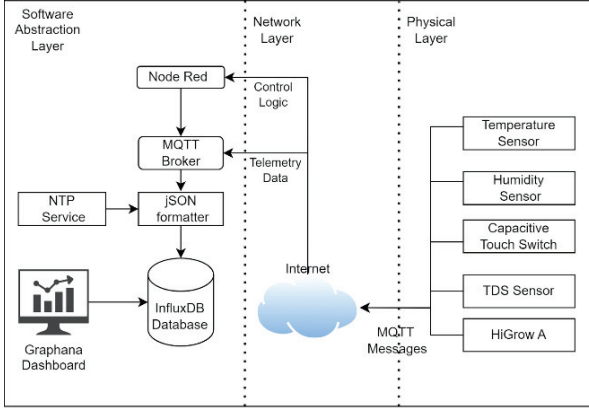


Fig. 8. Architectural diagram of the proposed system.

B. Deployed Sensor Topology

The Sensor topology is arranged according to the hierarchical architecture described in section I.e. Fig. 9, as MQTT topics as classification criteria as described in section II.a

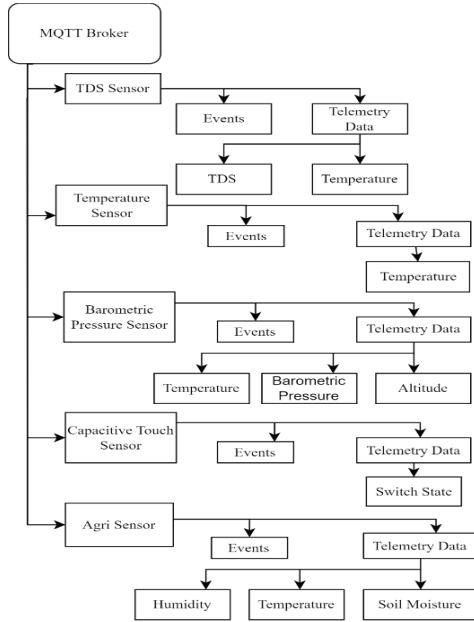


Fig. 9. Hierarchical MQTT Topic classification of the proposed system.

IV. PROPOSED MATHEMATICAL MODEL OF THE PROPAGATION DELAY OF THE PROPOSED SYSTEM

The latency for sending MQTT messages to the proposed system can be easily measured by sending a timestamp package from any of the sensors. If we send a timestamp or NTP clock count from the device periodically, we can find the periodic delay between each data reading. The ESP32 is a connected device, we can easily pull the timestamp using any NTP service library [9].

If we compare the timestamp value on the server end when we receive this packet, we can get T which is the time required for a specific data packet. For the N number of packets, we

will be presented with a matrix of size N with the delta(T)s of all N-packets [10].

As we add more sensors, theoretically these values should start getting larger and we will be able to observe how the number of sensors will increase the propagation delay.

Let us consider the periodic delay as P, which is a scalar value that denotes the interval at which we send the timestamp data from the sensor to the MQTT broker.

And

$$\bar{T}_{Tx} = [T_{Tx1} + T_{Tx2} + \dots + T_{Txn}] \quad (1)$$

Where N = the number of packets sent

On the Broker's end,

$$\bar{R}_{Rx1} = [R_{Rx1} + R_{Rx2} + \dots + R_{Rxn}] \quad (2)$$

Now, to get the time between sending the packet and receiving the packet on the MQTT broker, we can simply use linear algebra to obtain [11],

$$\delta \bar{T} = \bar{R}_{Rx1} - \bar{T}_{Tx} \quad (3)$$

Then the propagation delay,

$$\bar{T}_{pt} = \delta \bar{T} - P \quad (4)$$

II. HARDWARE DEVELOPMENT AND TESTING

To demonstrate the functionality of the proposed system four sensors were designed and fabricated while one sensor was procured off the shelf. In this section of the paper the design and fabrication process of these four sensor nodes are described in detail.

A. Sensor Node Design

1) Circuit Design

With our selection of peripherals in mind, we designed a simple breakout board for the ESP32 device with connectors for each type.

Our list of peripherals is 1) i2c display 2) i2c barometric Pressure sensor 3) Analog TDS sensor input 4) One Wire protocol temperature sensor input 5) Power Switch 6) Battery Connector. Fig 10 shows the schematic that was designed in Eagle.

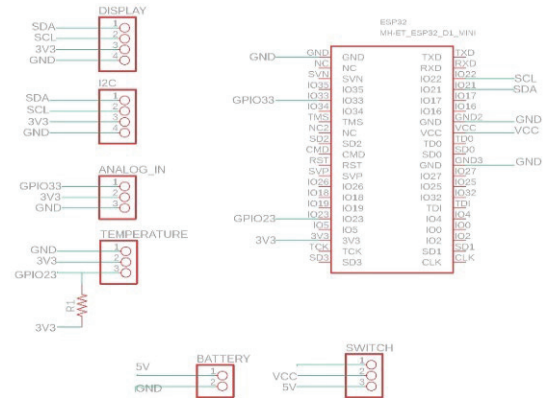


Fig. 10. Schematic of the Sensor Node Breakout Circuit.

2) PCB Design

Once the schematic was finalized, we designed the PCB, and Fig. 11 shows the board file. For each sensor we produced one of these PCB breakout boards and connected the sensors for interfacing.

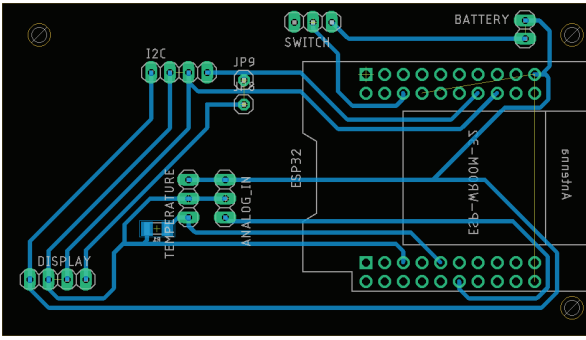


Fig. 11. PCB design for the Sensor node Breakout Circuit.

Every sensor module has an 18650 lithium-ion battery, TP4056 battery charger, and boost converter module attached to the battery connector which is shown in Fig. 12.

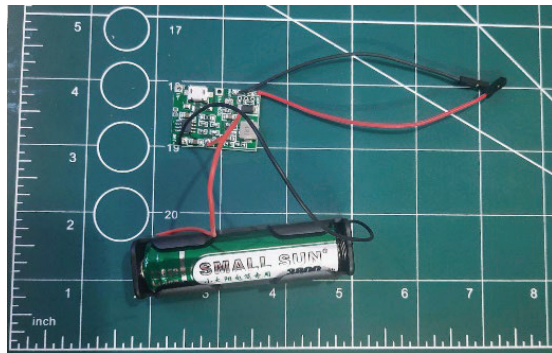


Fig. 12. Battery & Battery Management Peripherals

Until this stage of the fabrication, all sensor devices are alike. We then interface the individual sensors to each sensor node and will design and 3D print simple casings for them.

B. Sensor Node Fabrication

1) Total Dissolved Solids Sensors

The TDS sensor probes are first fed into an amplifier module developed by DFRobots. Which then gives us the amplified analog output which can be directly interfaced with a microcontroller. We also need to add a temperature sensor to this setup because the calculation of TDS requires temperature adjustment [12]. Fig. 13 demonstrates the wiring setup for the module.

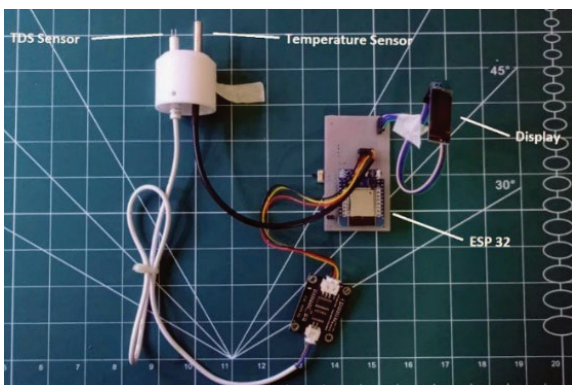


Fig. 13. Circuit Setup of TDS sensor node.

We then designed a simple waterproof case for the entire circuit setup which can be seen in Fig. 14.

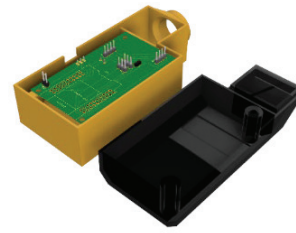


Fig. 14. Enclosure design of TDS sensor node.

The case was then 3D printed and fitted with the TDS sensor inside. Fig. 15 shows the fully assembled TDS sensor node.

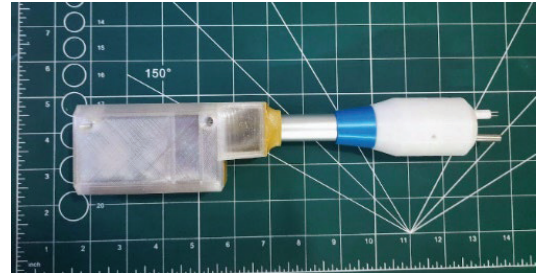


Fig. 15. Fabricated TDS sensor node.

2) Temperature Sensor

The temperature sensor we are using utilizes the 1-Wire protocol developed by Dallas Semiconductor Corp. The output needs to be pulled up to V_{cc} using a 10k resistor. The circuit setup for this sensor can be seen in Fig. 16

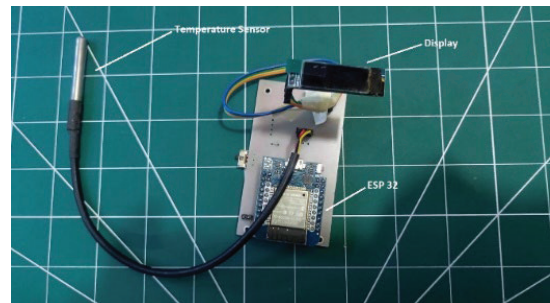


Fig. 16. Circuit Setup of the temperature sensor node.

We then designed a simple waterproof case for the entire circuit setup which can be seen in Fig. 17.

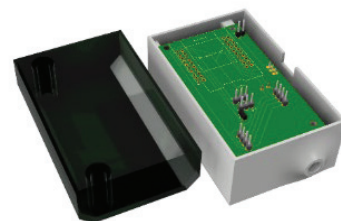


Fig. 17. Enclosure design of temperature sensor node.

The case was then 3D printed and fitted with the circuit setup inside. Fig. 18 shows the fully assembled Temperature Sensor.

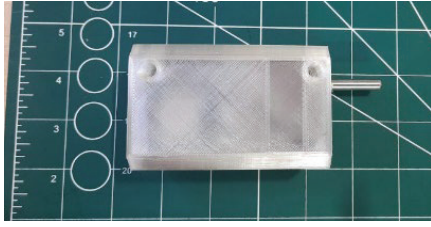


Fig. 18. Fabricated temperature sensor node.

3) Barometric Pressure

The barometric pressure sensor we're using was developed by Bosch. It utilizes the I²C protocol which uses the SDA and SCL data channels for interfacing. Fig. 19 demonstrates the circuit setup for the BMP180 Barometric pressure Sensor [13]. We simply connect the sensor to the I²C connector port we exposed in our PCB from Fig. 11.

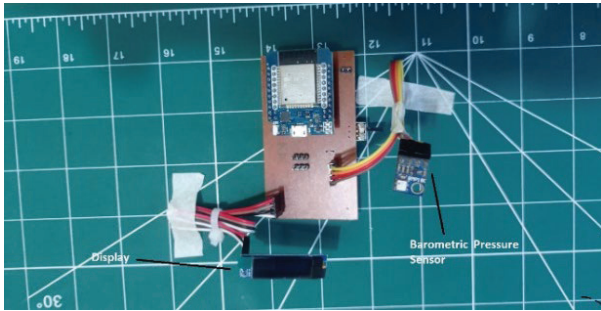


Fig. 19. Circuit Setup of TDS sensor node.

We then designed a simple waterproof case for the entire circuit setup which can be seen in Fig. 20.

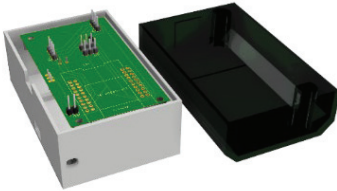


Fig. 20. Enclosure design of TDS sensor node.

The case was then 3D printed which is shown in Fig. 21.

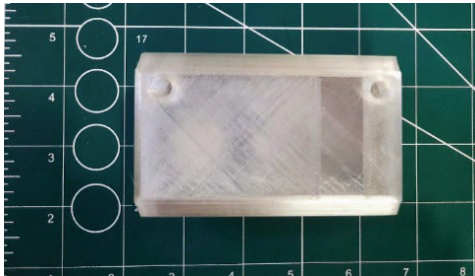


Fig. 21. Fabricated TDS sensor node.

4) Capacitive Touch

The capacitive touch module is based on the TTP229 IC. The module has a built-in RC circuit for switch debounce so the sensor can be connected to any digital pin of a microcontroller [14]. Fig. 22 shows the interfaced sensor setup

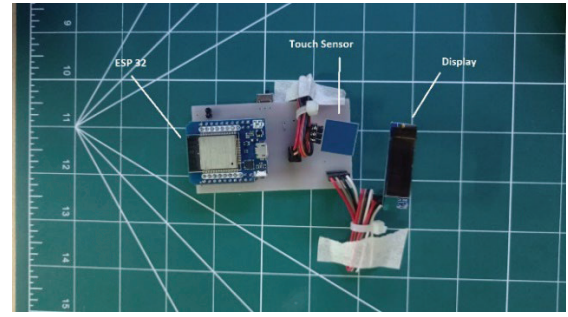


Fig. 22. Circuit Setup of TDS sensor node.

We then 3D model a simple case for it with the touch sensor being mounted on the lid (Fig. 23).

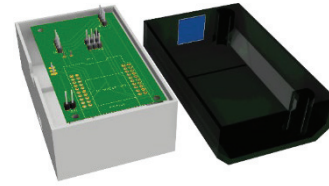


Fig. 23. Enclosure design of TDS sensor node.

The case was then 3D printed which looks similar to the barometric pressure sensor case, except for that it does not have a hole in its base.

5) Total Agricultural Soil Moisture & Humidity Sensor

This off-the-shelf sensor is widely used in agricultural embedded systems. It is known as the HiGrow sensor module which gives us Capacitive Soil Moisture data and humidity (Fig. 24). The humidity sensor attached is the DHT22 which also utilizes the One-Wire protocol [15].

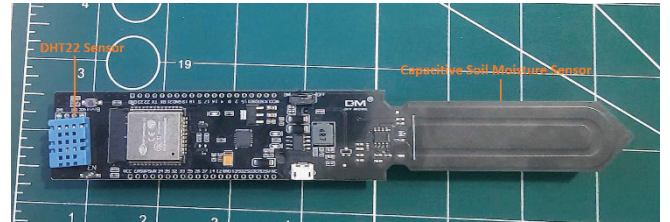


Fig. 24. Off The shelf HiGrow Sensor by DFRobots

III. RESULT & PERFORMANCE ANALYSIS

The Final output of the proposed system can be visually seen using the Grafana dashboard. All the sensor data can be seen as an aggregate or individually. They can also be queried using time as a range. Complex dashboards can be built very easily using the Grafana dashboard builder. Fig. 25 demonstrates such a dashboard.

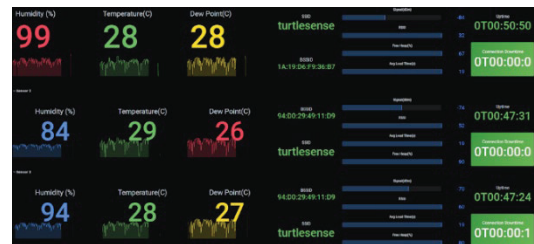


Fig. 25. Dashboard generated using Grafana using time series data

While simple dashboards for individual sensors can also be built using it which is demonstrated in Fig. 26.



Fig. 26. Dashboard for Individual Sensors

The MQTT protocol was designed for millions of concurrent sensors communicating without any loss of data. The scale at which we implemented our system cannot even start to reach the full potential of the system. While deployment, the maximum resource utilization we observed is described in Fig. 27.

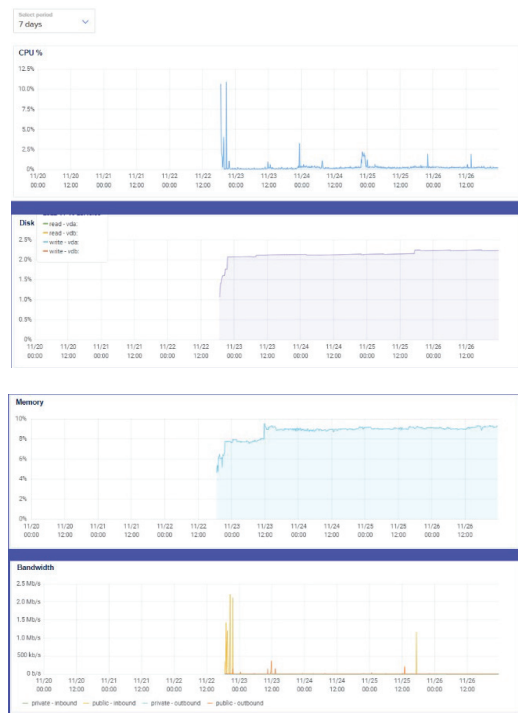


Fig. 27. VPS performance metrics of the deployed system.

Here we can see that we have not reached even 10% on any of the performance metrics for the VPS; CPU%, Memory Utilization, Disk Usage, and Disk Usage.

V. CONCLUSION

This research paper provides implementation guidelines for large-scale industrial IoT backend. The main objective of such a system is seamless integration and data acquisition. Future work on this domain can be done in the network layer where we can utilize 2G, 3G or 4G LTE communication to subscribe to our MQTT broker. MQTT over HTTP can be susceptible to network vulnerabilities for which we implemented MQTT over web sockets, but for narrowband or wideband GSM networks, MQTT over HTTP has to be implemented. Future work can be done in end-to-end encryption of MQTT and using QoS 3 to avoid transmitting redundant data.

REFERENCES

- [1] M. Klymash, T. Maksymyuk, S. Dumych, and O. Yaremko, "Designing the Industrial and Environmental Monitoring System based on the Internet of Things Architecture," *2018 IEEE 4th International Symposium on Wireless Systems within the International Conferences on Intelligent Data Acquisition and Advanced Computing Systems (IDAACS-SWS)*, 2018, pp. 187-190.
- [2] P. Bellavista and A. Zanni, "Towards better scalability for IoT-cloud interactions via combined exploitation of MQTT and CoAP," *2016 IEEE 2nd International Forum on Research and Technologies for Society and Industry Leveraging a better tomorrow (RTSI)*, 2016, pp. 1-6.
- [3] Naqvi, Syeda Noor Zehra, Sofia Yfantidou, and Esteban Zimányi. "Time series databases and influxdb." *Studienarbeit, Université Libre de Bruxelles 12* (2017).
- [4] K. Dewri, M. Hasan and M. R. Uddin, "IoT-based Energy Optimization and Demand Response System for Renewable Energy Integration," *2023 10th IEEE International Conference on Power Systems (ICPS)*, Cox's Bazar, Bangladesh, 2023, pp. 1-5.
- [5] M. H. Khan, M. R. Uddin, Z. Tasneem and K. M. Salim, "Power Factor Analysis of an Energy Efficient Single Phase VFD for Industrial Sewing Machines," *2019 5th International Conference on Advances in Electrical Engineering (ICAEE)*, 2019, pp. 800-803.
- [6] MQTT Version 5.0. Edited by Andrew Banks, Ed Briggs, Ken Borgendale, and Rahul Gupta. 07 March 2019. OASIS Standard. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>.
- [7] A. Amjad, F. Azam, M. W. Anwar and W. H. Butt, "A Systematic Review on the Data Interoperability of Application Layer Protocols in Industrial IoT," in *IEEE Access*, vol. 9, pp. 96528-96545, 2021, doi: 10.1109/ACCESS.2021.3094763.
- [8] S. B. Alam, S. S. Islam, M. R. Uddin and K. M. Salim, "48V Three-Phase Induction Motor Development for a Portable Industrial Vacuum Cleaner by using LV VFD Controller," *2020 11th International Conference on Electrical and Computer Engineering (ICECE)*, 2020, pp. 254-257.
- [9] M. N. Khan, F. S. Oishi, K. M. Mehzabin, M. H. Khan, M. R. Uddin and M. Hasan, "Solar powered IoT Based Remote Patient Monitoring System to Measure Vital Body Signs," *2023 15th International Conference on Intelligent Human-Machine Systems and Cybernetics (IHMSC)*, Hangzhou, China, 2023, pp. 159-162.
- [10] S. Haque, S. B. Khair, M. Sadman, S. Nadia, M. Shidujaman, M. R. Uddin, M. Hasan, "Automatic Product Sorting and Packaging System," *2023 15th International Conference on Intelligent Human-Machine Systems and Cybernetics (IHMSC)*, Hangzhou, China, 2023, pp. 163-167.
- [11] Liu X, Zhang T, Hu N, Zhang P, Zhang Y. The method of Internet of Things access and network communication based on MQTT. *Computer Communications*. 2020 Mar 1;153:169-76.
- [12] R. Parvin, R. Al Adnan, M. R. Uddin and M. Hasan, "Human-Machine Interface (HMI) Robotic Arm Controlled by Gyroscopically Acceleration," *2022 4th International Conference on Electrical, Computer & Telecommunication Engineering (ICECTE)*, Rajshahi, Bangladesh, 2022, pp. 1-4.
- [13] Meng X, Pang K, Di B, Li W, Wang Y, Zhang J, Xu Y. Road grade estimation for vehicle emissions modeling using electronic atmospheric pressure sensors. *Frontiers in Environmental Science*. 2023 Jan 4; 10:1051858.
- [14] S. T. Sarkar, J. A. Muhtaseem, M. R. Uddin and K. M. Salim, "Scroll Saw Machine Fabrication with High-Speed 3-Phase VFD Motor Drives for Photovoltaic Small Wooden Industrial Application of Off-grid Areas," *2021 2nd International Conference on Robotics, Electrical and Signal Processing Techniques (ICREST)*, 2021, pp. 394-397.
- [15] F. Z. Ha-mim, S. I. Aboni, M. A. Mim, M. R. Uddin and M. Hasan, "IoT-Based Automatic Seed Sowing and Plant Nutrition System," *2023 5th International Conference on Sustainable Technologies for Industry 5.0 (STI)*, Dhaka, Bangladesh, 2023, pp. 1-5.